03_marts

5. Marts

Marts are Helios's **consumption layer** — the only models the semantic layer (and therefore the agents) are allowed to read. Everything upstream (`src_ga4` → `stg_ga4__*` → `int_ga4__*`) exists to feed them; everything downstream (`semantic_layer.yaml` → semantic-mcp → the 7 agents) consumes them. A mart is the contract between analytics engineering and the product: if a number is in a mart, it is governed, tested, documented, and reconciled against raw. If it is not in a mart, the agents physically cannot reach it.

Marts are organized into three folders — **core** (the session/funnel spine + conformed dims), **finance** (orders + line items), and **growth** (decomposition rollups + retention cohorts) — matching the `models/marts/{core,finance,growth}/` tree and the per-folder materializations in `dbt_project.yml`.

5.1 Mart catalog

model	folder	layer	grain	primary key	materialization	purpose	feeds grain
`fct_sessions`	core	mart	session	`session_key`	incremental (`insert_overwrite`)	engagement + wide session dims + funnel reach	`fct_s`
`fct_funnel`	core	mart	session	`session_key`	incremental (`insert_overwrite`)	`reached_*` flags + `session_revenue` + wide dims; **primary session grain**	`fct_f`
`fct_daily_funnel`	core	mart	day × [channel_group, device_category, country, is_new_user]	`daily_funnel_key` (md5 of grain)	incremental (`insert_overwrite`)	additive step counts + revenue; Monitor/Decompose/eval feed	(rollup grain)
`dim_users`	core	mart	user_pseudo_id	`user_key`	table	first-touch attrs, new/returning (cookie grain)	dimens `fct_*`
`dim_items`	core	mart	item_id	`item_key`	table	item_name/brand/category(2..5)/price	dimens `fct_o`
`dim_channels`	core	mart	channel_group (10 rows)	`channel_key`	table	the 10 GA4 default channel groups + is_paid/is_organic/order	confor channe
`dim_date`	core	mart	day	`date_key`	table	conformed date spine 2020-11-01..2021-01-31	confor dim
`fct_orders`	finance	mart	transaction_id	`order_key`	table	gross/net/refund/shipping/tax + wide channel/device/date	`fct_o`
`fct_order_items`	finance	mart	transaction_id × item line	`order_item_key`	table	`item_revenue_in_usd`, quantity, item dims	`fct_o`
`fct_funnel_by_dim`	growth	mart	day × canonical dimension	`funnel_by_dim_key`	table	funnel rollup by canonical dims (decomposition input)	(decon input)
`fct_cohorts`	growth	mart	cohort_week × age_days	`cohort_key`	table	weekly acquisition cohorts (day_1/7/30 retention)	`fct_c`

The five base grains the semantic layer resolves against are exactly `fct_funnel`, `fct_sessions`, `fct_orders`, `fct_order_items`, and `fct_cohorts` (see `semantic_layer.yaml`); the other marts are conformed dimensions or pre-aggregated rollups that the agents read indirectly.

5.2 Why marts are WIDE (denormalized) facts

Every Helios fact is **denormalized** — it carries its own descriptive dimensions (`device_category`, `channel_group`, `country`, `is_new_user`, `event_date`) physically on the row rather than only foreign keys. This is deliberate and load-bearing for three reasons:

1. **The semantic layer slices without runtime joins.** When the Decompose agent asks `build_query('session_conversion_rate', dims=['device_category','channel_group'])`, semantic-mcp emits a single `GROUP BY` against `fct_funnel` — no join graph to plan, no join key cardinality surprises, no fan-out. The mix-vs-rate decomposition is one scan of one table.
2. **BigQuery rewards wide, not normalized.** BigQuery is a columnar scan engine; joins are comparatively expensive and column projection is nearly free. Denormalizing the handful of low-cardinality dims onto each fact costs trivial storage and removes the join from the hot path, which directly serves the **≤5 GiB/run byte budget** and **<5 min time-to-diagnosis** targets.
3. **Determinism and reproducibility.** A wide fact freezes the dimension values as they were at session time. There is no risk of a slowly-changing dimension re-classifying historical rows on the next run — the row is self-contained and idempotent under partition overwrite.

Surrogate keys (`session_key`, `order_key`, `channel_key`, `date_key`, `user_key`, `item_key`, `order_item_key`) still exist so the dims can be joined for enrichment and so `relationships` tests can enforce referential integrity, but the agents never need those joins at query time. `dim_channels`, `dim_date`, and `dim_users` are **conformed** — the same physical dimension is shared by `fct_sessions`, `fct_funnel`, `fct_orders`, and the daily rollups — so a slice by `channel_group` means the same thing in every fact, which is what lets the Decompose agent pivot any metric across the same canonical axes.

5.3 Incremental strategy (core facts)

The three session-grain core facts are large and append-shaped, so they use the production incremental pattern. The other marts (`dim_*`, finance, growth) are small enough to rebuild as `table` every run.

```
{{ config(
    materialized='incremental',
    incremental_strategy='insert_overwrite',
    partition_by={'field': 'event_date', 'data_type': 'date', 'granularity': 'day'},
    cluster_by=['device_category', 'channel_group'],
    require_partition_filter=true,
    on_schema_change='append_new_columns'
) }}
```

- **`insert_overwrite` + `partition_by(event_date)`** replaces *whole day partitions* atomically. A re-run over the same days produces byte-identical output (idempotent); a late-arriving or corrected `events_YYYYMMDD` shard reprocesses cleanly because the entire day is overwritten — no surrogate-key dedup, no MERGE, no drift.
- **3-day `is_incremental()` lookback** absorbs GA4's late event landing without reprocessing the full history:

  ```
  {% if is_incremental() %}
    where event_date ≥ date_sub(_dbt_max_partition, interval 3 day)
  {% endif %}
  ```

- **Partition pruning + clustering.** Every diagnosis scopes a date window, so `partition_by(event_date)` prunes to the touched days and `cluster_by(device_category, channel_group)` gives a second block-level filter on the two highest-traffic decomposition dimensions. `require_partition_filter=true` makes a date-less query *error* rather than full-scan.

Honesty on the static sample. The public dataset is frozen at 2020-11-01..2021-01-31, so no new shards ever land and source freshness is effectively N/A. In dev we therefore `dbt build --full-refresh` (3 months rebuilds in seconds and is the safe reset). The incremental config above is the **production** design — wired and correct so that the moment Helios points at a live daily GA4 export, the 3-day lookback and partition overwrite work without code changes. The static sample simply never exercises the `is_incremental()` branch.

5.4 Core marts

fct_sessions

- **Grain:** one row per session. **PK:** `session_key`.
- **Key columns:** `session_key`, `user_pseudo_id` (FK→`dim_users`), `event_date` (partition/FK→`dim_date`), `channel_group` (FK→`dim_channels`), `source`, `medium`, `landing_page`, `device_category`, `operating_system`, `browser`, `country`, `region`, `is_new_user`, `ga_session_number`, `event_count`, `engaged_session`, `engagement_time_msec`.
- **Materialization:** incremental `insert_overwrite` — large, append-shaped, day-partitioned.
- **Feeds semantic:** base grain `fct_sessions` for volume/engagement metrics (`sessions`, `engaged_sessions`, `engagement_rate`, the RPS denominator).

fct_funnel

- **Grain:** one row per session. **PK:** `session_key`. Joins **1:1** to `fct_sessions`.

- **Key columns:** the wide session dims (same as `fct_sessions`) + the monotonic `reached_*` flags (`reached_view_item` … `reached_purchase`) + `session_revenue`.
- **Materialization:** incremental `insert_overwrite`.
- **Feeds semantic:** the **primary session grain**. `session_conversion_rate`, `view_to_cart_rate`, all step rates, and the funnel session counts resolve here. Rates are computed as `SUM(numerator)/SUM(denominator)` from the additive `reached_*` flags after grouping — never stored, never an average of ratios.

fct_daily_funnel

- **Grain:** one row per (`event_date`, `channel_group`, `device_category`, `country`, `is_new_user`). **PK:** `daily_funnel_key`.
- **Key columns:** additive counts only — `sessions`, `view_item_sessions`, `add_to_cart_sessions`, `begin_checkout_sessions`, `add_shipping_info_sessions`, `add_payment_info_sessions`, `purchasing_sessions`, `transactions`, `revenue`.
- **Materialization:** incremental `insert_overwrite` (day-partitioned).
- **Feeds semantic:** the Monitor (time-series anomaly), Decompose (mix-shift), and eval feed. **Rates are NOT stored** — only additive counts — so they re-aggregate correctly across any slice. It aggregates `fct_funnel` (which carries `session_revenue`), *not* `int_ga4__funnel_steps`; dropping revenue here would break the eval's dollar-at-risk labels.

dim_users

- **Grain:** one row per `user_pseudo_id`. **PK:** `user_key`.
- **Key columns:** `first_touch_ts`, `first_touch_source`, `first_touch_medium`, `first_channel_group`, `total_sessions`, `total_revenue`, `is_purchaser`, `is_new_user`.
- **Materialization:** `table`. **Feeds semantic:** the user-grain dimension/denominator for ARPU; cookie-grain only (`user_id` is null), caveated by the Critic as a cookie approximation.

dim_items, dim_channels, dim_date

- **`dim_items`** — PK `item_key`; `item_name`, `item_brand`, `item_category(2..5)`, `price`. Conformed dimension for `fct_order_items`. `table` (with `snap_dim_items` providing SCD2 history on price/category).
- **`dim_channels`** — PK `channel_key`; exactly 10 rows, one per `channel_group`, with `is_paid`, `is_organic`, `channel_group_order`. The conformed channel dimension; tested with `accepted_values` of the 10 groups. `table`.
- **`dim_date`** — PK `date_key`; conformed date spine 2020-11-01..2021-01-31 with `week`, `month`, `quarter`, `day_of_week`, `is_weekend`. `table`.

5.5 Finance marts

fct_orders

- **Grain:** one row per `transaction_id` (deduped). **PK:** `order_key` = `transaction_id`.
- **Key columns:** `session_key` (FK→ `fct_sessions`), `user_pseudo_id` (FK→ `dim_users`), `event_date` (FK→ `dim_date`), `channel_group` (FK→ `dim_channels`), `device_category`, `order_ts`, `gross_revenue`, `refund_value_in_usd`, `net_revenue`, `shipping_value_in_usd`, `tax_value_in_usd`, `total_item_quantity`, `unique_items`.
- **Materialization:** `table` (small; full rebuild is cheap). **Feeds semantic:** base grain `fct_orders` for `revenue`, `gross_revenue`, `net_revenue`, `transactions`, `aov`, `items_per_transaction`.

fct_order_items

- **Grain:** one row per (`transaction_id`, item line). **PK:** `order_item_key` = md5(transaction_id + item_id + line ordinal).
- **Key columns:** `order_key` (FK→ `fct_orders`), `item_key` (FK→ `dim_items`), `item_name`, `item_category`, `quantity`, `item_revenue_in_usd`, `price_in_usd`, `coupon`.
- **Materialization:** `table`. **Feeds semantic:** base grain `fct_order_items` for item-revenue and category diagnoses.

5.6 Growth marts

fct_funnel_by_dim

- **Grain:** one row per (`event_date`, canonical dimension name/value). **PK:** `funnel_by_dim_key`.
- **Purpose:** a long-format funnel rollup keyed by `dimension_name` + `dimension_value` so the Decompose agent's mix-vs-rate decomposition can iterate over canonical dimensions uniformly. `table`. (Decomposition input; not a semantic base grain.)

fct_cohorts

- **Grain:** one row per (`cohort_week` , `age_days`). **PK:** `cohort_key` .
- **Key columns:** `cohort_week` , `age_days` , `cohort_size` , `active_users` , `retention_rate` .
- **Materialization:** `table` . **Feeds semantic:** base grain `fct_cohorts` for `day_1/7/30_retention` . Retention denominators are the fixed original cohort size (never the surviving population), so retention is monotonically non-increasing.

5.7 Marts best practices applied here

- **One mart per business entity.** `fct_funnel` =session, `fct_orders` =transaction, `fct_order_items` =order line, `fct_cohorts` =cohort×age. No mart mixes grains.
- **No BI-only logic in marts.** Rates, ratios, and derived metrics (`session_conversion_rate` , `aov` , RPS) live **only** in `semantic_layer.yaml` , computed `SUM(num)/SUM(den)` after grouping. Marts store additive primitives; presentation/derivation is the semantic layer's job.
- **Business logic lives upstream, exactly once.** Sessionization and `reached_*` flags live in the `int_ga4__*` keystones; channel grouping lives only in `channel_group_case()` . Marts assemble, they do not re-derive.
- **Mart → semantic grain mapping** is explicit and 1:1: `fct_funnel` , `fct_sessions` , `fct_orders` , `fct_order_items` , `fct_cohorts` are the only base grains the semantic layer resolves against.

5.8 fct_funnel.sql

```sql
-- models/marts/core/fct_funnel.sql
-- Session grain (PK session_key). Joins int_ga4__sessionized + int_ga4__funnel_steps 1:1.
-- Wide session dims + monotonic reached_* flags + session_revenue.
{{ config(
    materialized='incremental',
    incremental_strategy='insert_overwrite',
    partition_by={'field': 'event_date', 'data_type': 'date', 'granularity': 'day'},
    cluster_by=['device_category', 'channel_group'],
    require_partition_filter=true,
    on_schema_change='append_new_columns'
) }}

with sess as (
    select * from {{ ref('int_ga4__sessionized') }}
    {% if is_incremental() %}
      where date(timestamp_micros(session_start_micros))
            ≥ date_sub(_dbt_max_partition, interval 3 day)
    {% endif %}
),

steps as (
    select * from {{ ref('int_ga4__funnel_steps') }}
)

select
    -- keys / partition
    s.session_key,
    s.user_pseudo_id,
    date(timestamp_micros(s.session_start_micros))      as event_date,    -- partition col
    -- wide, denormalized session dimensions (no runtime join needed downstream)
    s.channel_group,
    s.source,
    s.medium,
    s.landing_page,
    s.device_category,
    s.operating_system,
    s.browser,
    s.country,
    s.region,
    (s.ga_session_number = 1)                           as is_new_user,
    s.engaged_session,
    -- monotonic, max-downstream funnel flags (sessions ≥ reached_view_item ≥ ... ≥ reached_purchase)
    st.did_session_start,
    st.reached_view_item,
    st.reached_add_to_cart,
    st.reached_begin_checkout,
    st.reached_add_shipping_info,
    st.reached_add_payment_info,
    st.reached_purchase,
    -- per-session deduped revenue (one purchase_revenue per distinct transaction_id)
    coalesce(st.session_revenue, 0.0)                   as session_revenue
from sess s
join steps st using (session_key)   -- 1:1; both are session-grained on session_key
```

5.9 fct_daily_funnel.sql

```
-- models/marts/core/fct_daily_funnel.sql
-- Additive daily funnel: COUNT(DISTINCT IF(reached_*, session_key, NULL)) per step + revenue.
-- Grouped by event_date x canonical dims. RATES ARE NOT STORED (semantic layer computes them).
-- Aggregates fct_funnel (carries session_revenue) -- NOT int_ga4__funnel_steps.
{{ config(
    materialized='incremental',
    incremental_strategy='insert_overwrite',
    partition_by={'field': 'event_date', 'data_type': 'date', 'granularity': 'day'},
    require_partition_filter=true
) }}

with funnel as (
    select * from {{ ref('fct_funnel') }}
    {% if is_incremental() %}
      where event_date ≥ date_sub(_dbt_max_partition, interval 3 day)
    {% endif %}
)

select
    -- surrogate PK over the grain columns
    {{ dbt_utils.generate_surrogate_key(
         ['event_date','channel_group','device_category','country','is_new_user']) }}
                                                                    as daily_funnel_key,
    event_date,
    channel_group,
    device_category,
    country,
    is_new_user,
    -- additive step counts (re-aggregatable across any slice)
    count(distinct session_key)                                     as sessions,
    count(distinct if(reached_view_item,         session_key, null))    as view_item_sessions,
    count(distinct if(reached_add_to_cart,       session_key, null))    as add_to_cart_sessions,
    count(distinct if(reached_begin_checkout,    session_key, null))    as begin_checkout_sessions,
    count(distinct if(reached_add_shipping_info, session_key, null))    as add_shipping_info_sessions,
    count(distinct if(reached_add_payment_info,  session_key, null))    as add_payment_info_sessions,
    count(distinct if(reached_purchase,          session_key, null))    as purchasing_sessions,
    count(distinct if(reached_purchase and session_revenue > 0, session_key, null)) as transactions,
    sum(session_revenue)                                            as revenue
from funnel
group by event_date, channel_group, device_category, country, is_new_user
-- NOTE: session_conversion_rate etc. are NOT selected here. They are derived in
-- semantic_layer.yaml as SUM(purchasing_sessions)/SUM(sessions) after grouping.
```

5.10 fct_orders.sql

```
-- models/marts/finance/fct_orders.sql
-- One deduped row per transaction_id (PK order_key). Wide channel/device/date dims.
-- gross/net/refund/shipping/tax. Small fact → full table rebuild.
{{ config(materialized='table') }}

with purch as (
    -- collapse duplicate purchase rows for one transaction_id (GA4 emits retries/multi-stream)
    select
        ecommerce.transaction_id                            as order_key,
        {{ sessionize() }}                                  as session_key,
        user_pseudo_id,
        timestamp_micros(event_timestamp)                   as order_ts,
        date(timestamp_micros(event_timestamp))             as event_date,
        any_value(ecommerce.purchase_revenue_in_usd)        as gross_revenue,
        any_value(ecommerce.refund_value_in_usd)            as refund_value_in_usd,
        any_value(ecommerce.shipping_value_in_usd)          as shipping_value_in_usd,
        any_value(ecommerce.tax_value_in_usd)               as tax_value_in_usd,
        any_value(ecommerce.total_item_quantity)            as total_item_quantity,
        any_value(ecommerce.unique_items)                   as unique_items
    from {{ source('src_ga4','events') }}
    where event_name = 'purchase'
      and ecommerce.transaction_id is not null              -- exclude untagged purchases
      and _table_suffix between '20201101' and '20210131'   -- shard prune the static sample
    group by order_key, session_key, user_pseudo_id, order_ts, event_date
),

dims as (
    -- pull the wide session dims so fct_orders slices without a runtime join
    select session_key, channel_group, device_category, country
    from {{ ref('fct_sessions') }}
```

```
)

select
    p.order_key,
    p.session_key,
    p.user_pseudo_id,
    p.event_date,
    p.order_ts,
    coalesce(d.channel_group, 'Other')                      as channel_group,  -- wide
    d.device_category,
    d.country,
    p.gross_revenue,
    coalesce(p.refund_value_in_usd, 0.0)                    as refund_value_in_usd,
    p.gross_revenue - coalesce(p.refund_value_in_usd, 0.0)  as net_revenue,
    coalesce(p.shipping_value_in_usd, 0.0)                  as shipping_value_in_usd,
    coalesce(p.tax_value_in_usd, 0.0)                       as tax_value_in_usd,
    p.total_item_quantity,
    p.unique_items
from purch p
left join dims d using (session_key)   -- left join: keep orders even if session dim is missing
```

5.11 dim_channels.sql

```
-- models/marts/core/dim_channels.sql
-- One row per channel_group (exactly 10). is_paid / is_organic / channel_group_order.
-- channel_group strings come from the SINGLE SOURCE OF TRUTH macro (channel_group_case()),
-- enumerated here once as the conformed dimension.
{{ config(materialized='table') }}

with channels as (
    select * from unnest([
        struct('Direct'         as channel_group, false as is_paid, false as is_organic,  1 as channel_group_order),
        struct('Organic Search' as channel_group, false as is_paid, true  as is_organic,  2 as channel_group_order),
        struct('Paid Search'    as channel_group, true  as is_paid, false as is_organic,  3 as channel_group_order),
        struct('Display'        as channel_group, true  as is_paid, false as is_organic,  4 as channel_group_order),
        struct('Paid Social'    as channel_group, true  as is_paid, false as is_organic,  5 as channel_group_order),
        struct('Organic Social' as channel_group, false as is_paid, true  as is_organic,  6 as channel_group_order),
        struct('Email'          as channel_group, false as is_paid, true  as is_organic,  7 as channel_group_order),
        struct('Affiliates'     as channel_group, true  as is_paid, false as is_organic,  8 as channel_group_order),
        struct('Referral'       as channel_group, false as is_paid, true  as is_organic,  9 as channel_group_order),
        struct('Other'          as channel_group, false as is_paid, false as is_organic, 10 as channel_group_order)
    ])
)

select
    to_hex(md5(channel_group))   as channel_key,   -- surrogate PK
    channel_group,
    is_paid,
    is_organic,
    channel_group_order
from channels
```

5.12 fct_cohorts.sql

```
-- models/marts/growth/fct_cohorts.sql
-- Weekly acquisition cohort x age_days for retention (day_1/7/30).
-- cohort_week = week of first touch; age_days = days since cohort start a user was active.
-- Denominator = fixed original cohort size → retention monotonically non-increasing.
{{ config(materialized='table') }}

with first_touch as (
    -- one row per user: the week they were acquired
    select
        user_pseudo_id,
        date_trunc(min(event_date), week(monday))   as cohort_week
    from {{ ref('fct_sessions') }}
    group by user_pseudo_id
),

activity as (
    -- every (user, day) they had a session, with days-since-acquisition
    select
        s.user_pseudo_id,
        ft.cohort_week,
        date_diff(s.event_date, ft.cohort_week, day) as age_days
    from {{ ref('fct_sessions') }} s
```

```
    join first_touch ft using (user_pseudo_id)
    where s.event_date >= ft.cohort_week
),

cohort_size as (
    select cohort_week, count(distinct user_pseudo_id) as cohort_size
    from first_touch
    group by cohort_week
),

active_by_age as (
    select
        cohort_week,
        age_days,
        count(distinct user_pseudo_id) as active_users
    from activity
    group by cohort_week, age_days
)

select
    {{ dbt_utils.generate_surrogate_key(['a.cohort_week','a.age_days']) }} as cohort_key,
    a.cohort_week,
    a.age_days,
    cs.cohort_size,
    a.active_users,
    safe_divide(a.active_users, cs.cohort_size)  as retention_rate
from active_by_age a
join cohort_size cs using (cohort_week)
```

5.13 schema.yml — core / finance / growth

The mart tests enforce the three guarantees the agents rely on: **uniqueness** of every grain PK, **referential integrity** of every conformed FK (`relationships`), and the **closed set** of channel groups (`accepted_values` = exactly 10). `persist_docs` pushes descriptions to BigQuery so the catalog is self-documenting.

```
# models/marts/core/core__schema.yml
version: 2

models:
  - name: fct_funnel
    description: "One row per session (PK session_key). Monotonic reached_* flags + session_revenue + wide session dims. Primary session grain for the
      semantic layer."
    config:
      contract: {enforced: true}     # column-level contract on the primary grain
    columns:
      - name: session_key
        tests: [unique, not_null]
      - name: event_date
        tests: [not_null]
      - name: channel_group
        tests:
          - not_null
          - relationships: {to: ref('dim_channels'), field: channel_group}
      - name: user_pseudo_id
        tests:
          - relationships: {to: ref('dim_users'), field: user_pseudo_id}
      - name: reached_purchase
        tests: [not_null]
      - name: session_revenue
        tests:
          - not_null
          - dbt_utils.accepted_range: {min_value: 0}
    # singular tests assert_funnel_monotonicity + assert_session_conversion_rate_bounds
    # live in tests/ and guard the reached_* invariant and 0≤cvr≤1.

  - name: fct_sessions
    description: "One row per session (PK session_key). Engagement + wide session dims + funnel reach."
    columns:
      - name: session_key
        tests: [unique, not_null]
      - name: channel_group
        tests:
          - relationships: {to: ref('dim_channels'), field: channel_group}
      - name: engaged_session
        tests: [not_null]

  - name: fct_daily_funnel
```

```yaml
    description: "Additive daily funnel counts + revenue by event_date x channel/device/country/is_new_user. Rates NOT stored. Aggregates fct_funnel."
    columns:
      - name: daily_funnel_key
        tests: [unique, not_null]
      - name: channel_group
        tests:
          - relationships: {to: ref('dim_channels'), field: channel_group}
      - name: sessions
        tests:
          - not_null
          - dbt_utils.accepted_range: {min_value: 0}
    tests:
      # purchasing_sessions can never exceed sessions (monotonicity at the rollup grain)
      - dbt_utils.expression_is_true:
          expression: "purchasing_sessions ≤ sessions"

  - name: dim_channels
    description: "Conformed channel dimension. Exactly 10 GA4 default channel groups."
    columns:
      - name: channel_key
        tests: [unique, not_null]
      - name: channel_group
        tests:
          - not_null
          - unique
          - accepted_values:
              values: ['Direct','Organic Search','Paid Search','Display',
                       'Paid Social','Organic Social','Email','Affiliates',
                       'Referral','Other']

  - name: dim_users
    description: "One row per user_pseudo_id (PK user_key). Cookie-grain first-touch attrs; user_id null."
    columns:
      - name: user_key
        tests: [unique, not_null]
      - name: user_pseudo_id
        tests: [unique, not_null]

  - name: dim_date
    description: "Conformed date spine 2020-11-01..2021-01-31."
    columns:
      - name: date_key
        tests: [unique, not_null]

  - name: dim_items
    description: "One row per item_id (PK item_key). item_name/brand/category(2..5)/price."
    columns:
      - name: item_key
        tests: [unique, not_null]
```

```yaml
# models/marts/finance/finance__schema.yml
version: 2

models:
  - name: fct_orders
    description: "One deduped row per transaction_id (PK order_key). gross/net/refund/shipping/tax + wide channel/device/date."
    columns:
      - name: order_key
        tests: [unique, not_null]
      - name: session_key
        tests:
          - relationships: {to: ref('fct_sessions'), field: session_key}
      - name: channel_group
        tests:
          - relationships: {to: ref('dim_channels'), field: channel_group}
      - name: gross_revenue
        tests:
          - not_null
          - dbt_utils.accepted_range: {min_value: 0}
      - name: net_revenue
        tests:
          - dbt_utils.accepted_range: {min_value: 0}
    tests:
      - revenue_reconciles:           # custom generic test: mart revenue == raw to the cent
          column_name: gross_revenue
          tolerance: 0

  - name: fct_order_items
    description: "One row per (transaction_id, item line) (PK order_item_key). item_revenue_in_usd, quantity, item dims."
```

```yaml
    columns:
      - name: order_item_key
        tests: [unique, not_null]
      - name: order_key
        tests:
          - relationships: {to: ref('fct_orders'), field: order_key}
      - name: item_key
        tests:
          - relationships: {to: ref('dim_items'), field: item_key}
      - name: item_revenue_in_usd
        tests:
          - dbt_utils.accepted_range: {min_value: 0}
```

```yaml
# models/marts/growth/growth__schema.yml
version: 2

models:
  - name: fct_funnel_by_dim
    description: "Funnel rollup by canonical dimension (long format) for the Decompose mix-vs-rate input."
    columns:
      - name: funnel_by_dim_key
        tests: [unique, not_null]
      - name: dimension_name
        tests:
          - accepted_values:
              values: ['channel_group','device_category','country','is_new_user','landing_page']

  - name: fct_cohorts
    description: "Weekly acquisition cohort x age_days for day_1/7/30 retention."
    columns:
      - name: cohort_key
        tests: [unique, not_null]
      - name: cohort_week
        tests: [not_null]
      - name: retention_rate
        tests:
          - dbt_utils.accepted_range: {min_value: 0, max_value: 1}   # fixed-denominator retention in [0,1]
```

Together these marts are the foundation of the product: wide, conformed, grain-explicit facts that the semantic layer slices without joins, reconciled to raw to the cent, and tested for uniqueness, referential integrity, and the closed 10-channel vocabulary — so the agents compose governed metrics over them and never reach raw.